

The Hong Kong Polytechnic University

Department of Electronic and Information Engineering

EIE2111 Lab 2: Introduction to Classes and Objects

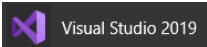
(Deadline for Submission: Check the course information)

Introduction

This laboratory exercise is designed to give you hand-on experience in Classes and Objects.

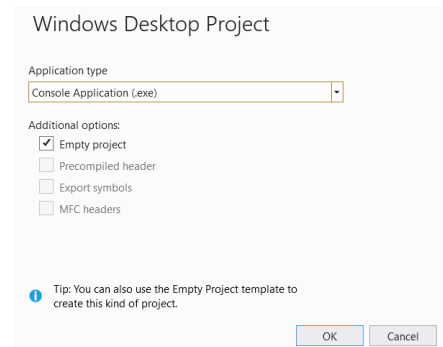
Part A: Develop your first class, **GradeBook**.

1. Run the application software “Microsoft Visual Studio 2019”.

- Choose and click “Visual Studio 2019”  from the program menu.

2. Create a new project called Lab02PartA.

- Select C++ under the “All languages” and click “Windows Desktop Wizard”.
- Click the option “New” and then “Project”.
- Type “Lab02PartA” in the field “Name” and then click the browse button “...” to select the storage location e.g. “Desktop”.
- Click the button “Create”. Select “Console Application” and “Empty Project”. Then, click the button “OK” (see the figure on the right hand side).



3. Create a C++ program file called Lab02PartA.cpp.

- Go to the window “Solution Explorer” and right click the folder “Source Files”.
- Click the option “Add” and then “New Item ...”.
Select “C++ file (.cpp)”, type “Lab02PartA.cpp” in the field “Name” and then click the button “Add”.
- Type the following statements in the window “Lab02PartA.cpp”:

```
//Lab02PartA.cpp
//Name: <Replace with your Student Name>

#include <iostream>
#include <string>
using namespace std;

class GradeBook
{
public:
    void displayMessage(string courseName)
    {
        cout << "Welcome to the grade book for \n" << courseName << "!" << endl;
    }
};
```

```

int main()
{
    string nameOfCourse;
    GradeBook myGradeBook;

    cout << "Please enter the course name:" << endl;
    getline(cin, nameOfCourse);
    cout << endl;
    myGradeBook.displayMessage(nameOfCourse);
}

```

4. Compile the source file.

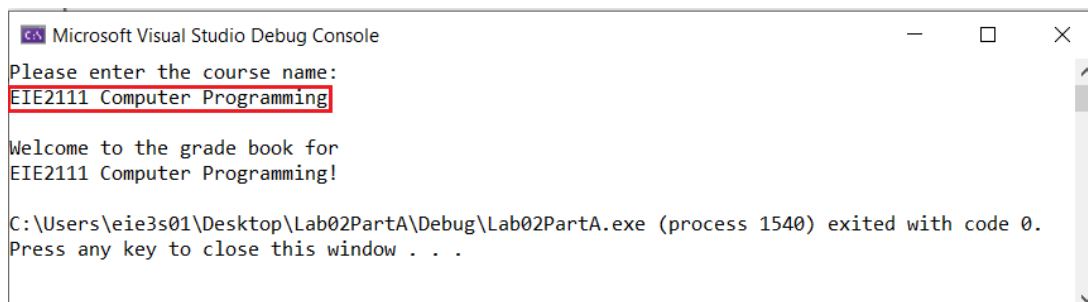
- Go to the main menu. Click the option “Build” and then “Build Solution”. If there is no error, you should find the following message on the window “Output” (*Fix any errors before you execute the program; alternatively, you may choose the option “Build” and then “Rebuild Solution” to delete all compiled files and compile the whole project again.*):

```

===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

```

- Go to the main menu. Click the option “Debug” and then “Start Without Debugging” (*or Ctrl + F5*). Then, type the highlighted text below and press “Enter”. You should find that the following window comes out to the screen:



5. Create a C++ header file called GradeBook.h.

- Go to the window “Solution Explorer” and right click the folder “Header Files”.
- Click the option “Add” and then “New Item ...”.
- Select “Header File (.h)”, type “GradeBook.h” in the field “Name” and then click the button “Add”.
- **Cut** and paste the following statements from “Lab02PartA.cpp” to “GradeBook.h”:

```

#include <string>
using namespace std;

class GradeBook
{
public:
    void displayMessage(string courseName)
    {
        cout << "Welcome to the grade book for \n" << courseName << "!" << endl;
    }
};

```

- In “Lab02PartA.cpp”, add the following statement after “#include <iostream>”:

```

#include "GradeBook.h"

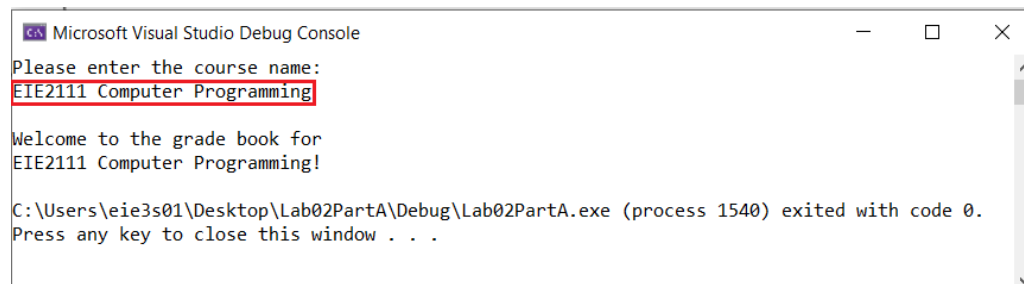
```

Compile the source file.

- Go to the main menu. Click the option “Build” and then “Build Solution”. If there is no error, you should find the following message on the window “Output” (*Fix any errors before you execute the program; alternatively, you may choose the option “Build” and then “Rebuild Solution” to delete all compiled files and compile the whole project again.*):

```
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

- Go to the main menu. Click the option “Debug” and then “Start Without Debugging” (or *Ctrl + F5*). You should find that the following window comes out to the screen:

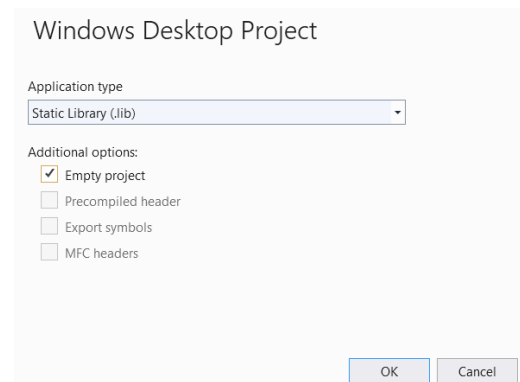


Part B: Develop your first static library, GradeBook .

1. Run the application software “Microsoft Visual Studio 2019”.
 - Choose and click “Visual Studio 2019” from the program menu.

2. Create a new project called GradeBook.

- Select C++ under the “All languages” and click “Windows Desktop Wizard”.
- Click the option “New” and then “Project”.
- Type “GradeBook” in the field “Name” and then click the browse button “...” to select the storage location e.g. “Desktop”.
- Click the button “Create”.
- Select “Static Library” and “Empty Project”. Then, click the button “OK” (see the figure on the right hand side).



3. Create a C++ header file called GradeBook.h.

- Go to the window “Solution Explorer” and right click the folder “Header Files”.
- Click the option “Add” and then “New Item ...”.
- Select “Header File (.h)”, type “GradeBook.h” in the field “Name” and then click the button “Add”.
- Type the following statements in the window “GradeBook.h”:

```
//GradeBook.h
#include <string>
using std::string;

class GradeBook
```

```

{
public:
    GradeBook(string);
    void setCourseName(string);
    string getCourseName();
    void displayMessage();

private:
    string courseName;
};

```

4. Compile the header file.

- Go to the main menu. Click the option “Build” and then “Build Solution”. If there is no error, you should find the following message on the window “Output” (*Fix any errors before you execute the program; alternatively, you may choose the option “Build” and then “Rebuild Solution” to delete all compiled files and compile the whole project again.*):

```

===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

```

5. Create a C++ source file called GradeBook.cpp.

- Go to the window “Solution Explorer” and right click the folder “Header Files”.
- Click the option “Add” and then “New Item ...”.
- Select “C++ file (.cpp)”, type “GradeBook.cpp” in the field “Name” and then click the button “Add”.
- Type the following statements in the window “GradeBook.cpp”:

```

//GradeBook.cpp
//Name: <Replace with your student name>
//Student ID: <Replace with your student ID>

#include <iostream>
using std::cout;
using std::endl;

#include "GradeBook.h"

GradeBook::GradeBook(string name)
{
    setCourseName(name);
}
void GradeBook::setCourseName(string name)
{
    courseName = name;
}

string GradeBook::getCourseName()
{
    return courseName;
}

void GradeBook::displayMessage()
{
    cout << "Welcome to the gradebook for\n" << getCourseName() << "!" <<
endl;
}

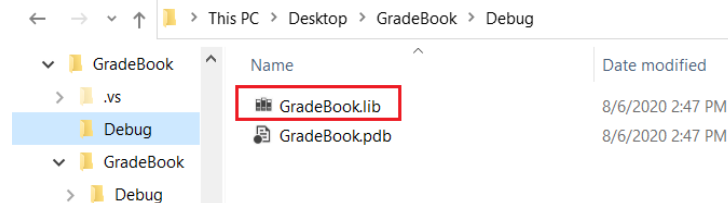
```

6. Compile the source file.

- Go to the main menu. Click the option “Build” and then “Build Solution”. If there is no error, you should find the following message on the window “Output” (*Fix any errors before you execute the program; alternatively, you may choose the option “Build” and then “Rebuild Solution” to delete all compiled files and compile the whole project again.*):

===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

Go to the project directory and then the sub-directory “Debug”, you will find that you create an object file called `GradeBook.lib` (see the figure below). We call the object file as a static library.



7. Create a new project called `GradeBookTest`.

- Select C++ under the “All languages” and click “Windows Desktop Wizard”.
- Click the option “New” and then “Project”.
- Type “`GradeBookTest`” in the field “Name” and then click the browse button “...” to select the storage location e.g. “Desktop”.
- Click the button “Create”. Select “Console Application” and “Empty Project”. Then, click the button “OK”.

8. Create a C++ source file called `GradeBookTest.cpp`.

- Go to the window “Solution Explorer” and right click the folder “Source Files”.
- Click the option “Add” and then “New Item ...”.
- Select “C++ File (.cpp)”, type “`GradeBookTest.cpp`” in the field “Name” and then click the button “Add”.
- Type the following statements in the window “`GradeBookTest.cpp`”:

```
//GradeBookTest.cpp
//Name: <Replace by your student name>
//Student ID: <Replace by your student ID>

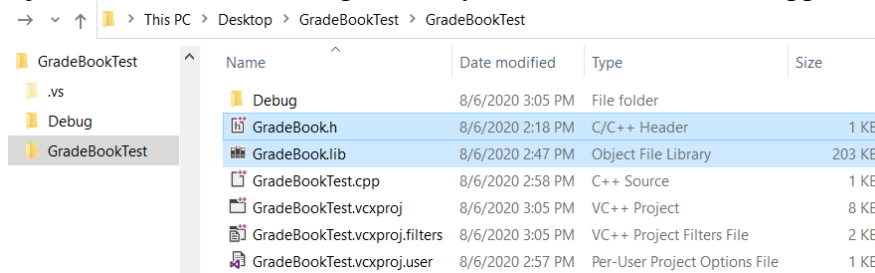
#include <iostream>
using std::cout;
using std::endl;

#include "GradeBook.h"

int main()
{
    GradeBook gradeBook1("CS101 Introduction to C++ Programming");
    GradeBook gradeBook2("CS102 Data Structures in C++");

    cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()
         << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
         << endl;
    return 0;
}
```

9. Use “File Explorer” to copy the files “GradeBook.h” and “GradeBook.lib” from previous GradeBook project folder to the working directory with “GradeBookTest.cpp”.



10. Add these two files into the project “GradeBookTest”.

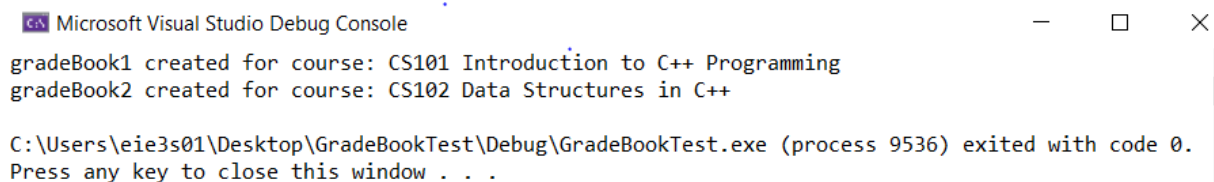
- Go to the window “Solution Explorer” and right click the folder “Source Files”.
- Click the option “Add” and then “Existing Item ...”.
- Select “GradeBook.h” and “GradeBook.lib” and then click the button “Add”.

11. Compile and execute the program.

- Go to the main menu. Click the option “Build” and then “Build Solution”. If there is no error, you should find the following message on the window “Output” (*Fix any errors before you execute the program; alternatively, you may choose the option “Build” and then “Rebuild Solution” to delete all compiled files and compile the whole project again.*):

```
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
```

- Go to the main menu. Click the option “Debug” and then “Start Without Debugging” (*or Ctrl + F5*). You should find that the following window comes out to the screen:



Part C: Develop your C++ programs with classes and objects.

Create a class called “Account”

1. Create a header file `Account.h` which includes a class called `Account`.
2. A bank uses the class `Account` to represent customers’ bank accounts. Your class should include one data member of type `int` to represent the account balance. Your class should provide a constructor that receives an initial balance and uses it to initialize the data member. The constructor should validate the initial balance to ensure that it is greater than or equal to 0. If not, the balance should be set to 0 and the constructor should display an error message, indicating that the initial balance was invalid. The class should provide three member functions. Member function `credit` should add an amount to the current balance. Member function `debit` should withdraw money from the `Account` and should ensure that the debit amount does not exceed the `Account`’s balance. If it does, the balance should be left unchanged and the function should print a message indicating "Debit amount exceeded account balance.". Member function `getBalance` should return the current balance.

Create a class called “Invoice”

3. Create a header file `Invoice.h` which includes a class called `Invoice`.
4. A hardware core uses the class `Invoice` to represent an invoice for an item sold at the store. An `Invoice` should include four pieces of information as data members — a part number (type `string`), a part description (type `string`), a quantity of the item being purchased (type `int`) and a price per item (type `int`). Your class should have a constructor that initializes the four data members. Provide a *set* and a *get* function for each data member. In addition, provide a member function named `getInvoiceAmount` that calculates the invoice amount (i.e., multiplies the quantity by the price per item), then returns the amount as an `int` value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.
5. You have to create two static libraries for two classes `Account` and `Invoice` for the test program in step 6.
6. The following main program can be used to test two classes `Account` and `Invoice`:

```
//Name: <Replace with your student name>
//Student ID: <Replace with your student ID>

#include <iostream>
using std::cout;
using std::endl;

#include "Account.h"
#include "Invoice.h"

int main()
{
    Account myAccount1(100);
    Account myAccount2(-10);

    cout << "myAccount1: Balance = $" << myAccount1.getBalance() << endl;
    cout << endl;
    cout << "Deposit $100 into the account." << endl;
    myAccount1.credit(100);
    cout << "myAccount1: Balance = $" << myAccount1.getBalance() << endl;
    cout << endl;
    cout << "Withdraw $500 from the account." << endl;
    myAccount1.debit(500);
    cout << "myAccount1: Balance = $" << myAccount1.getBalance() << endl;
    cout << endl;
    cout << "Withdraw $50 from the account." << endl;
    myAccount1.debit(50);
    cout << "myAccount1: Balance = $" << myAccount1.getBalance() << endl;
    cout << endl;

    Invoice myInvoice("12345", "Hammer", 100, 5);

    cout << "Part number: " << myInvoice.getPartNumber() << endl;
    cout << "Part description: " << myInvoice.getPartDescription() << endl;
    cout << "Quantity: " << myInvoice.getQuantity() << endl;
    cout << "Price per item: $" << myInvoice.getPricePerItem() << endl;
    cout << "Invoice amount: $" << myInvoice.getInvoiceAmount() << endl;
    cout << endl;

    myInvoice.setPartNumber( "123456" );
    myInvoice.setPartDescription( "Saw" );
    myInvoice.setQuantity( -5 );
```

```

myInvoice.setQuantity( 5 );
myInvoice.setPricePerItem( -10 );
myInvoice.setPricePerItem( 10 );
cout << endl;

cout << "After modification, " << endl;
cout << "Part number: " << myInvoice.getPartNumber() << endl;
cout << "Part description: " << myInvoice.getPartDescription() << endl;
cout << "Quantity: " << myInvoice.getQuantity() << endl;
cout << "Price per item: $" << myInvoice.getPricePerItem() << endl;
cout << "Invoice amount: $" << myInvoice.getInvoiceAmount() << endl;

return 0;
}

```

The output of the above test program is listed below:

```

C:\Windows\system32\cmd.exe
Error: Initial balance cannot be negative.
myAccount1: Balance = $100

Deposit $100 into the account.
myAccount1: Balance = $200

Withdraw $500 from the account.
Debit amount exceeded account balance.
myAccount1: Balance = $200

Withdraw $50 from the account.
myAccount1: Balance = $150

Part number: 12345
Part description: Hammer
Quantity: 100
Price per item: $5
Invoice amount: $500

quantity cannot be negative.
quantity set to 0.
pricePerItem cannot be negative.
pricePerItem set to 0.

After modification,
Part number: 123456
Part description: Saw
Quantity: 5
Price per item: $10
Invoice amount: $50
Press any key to continue . . .

```

Instructions

- You are required to submit ALL your C++ programs (i.e. whole project folders with source code inside; each project is stored inside a single folder which is created by Microsoft Visual Studio 2019) in Part A, B and C to Blackboard. Use Zip or 7-Zip to compress all of them into a single file. Please add your student name & ID as a comment (e.g. *//Student Name/ID: XXX XXX*) at the beginning of each .cpp file. Remember to **close** any output window and Visual Studio **before** compression and submission.
- The deadline of the submission: **Check the course information**.

Richard Pang
 Lawrence Cheung
 August 2020